

# Sistema de Votação Eletrônica

## Multi-sessão com Supabase

|                      |                                    |
|----------------------|------------------------------------|
| <b>Backend:</b>      | Supabase (PostgreSQL + Auth + RPC) |
| <b>Frontend:</b>     | React 18 + Vite + Tailwind         |
| <b>Deploy:</b>       | Docker (LAN) / Vercel (Internet)   |
| <b>Senha padrão:</b> | admin123                           |

## Guia de Instalação Completo

Servidor local • Internet • Hospedagem estática

## Sumário

1. Visão geral da arquitetura
2. Requisitos do sistema
3. Opção A — Supabase Cloud (internet)
4. Opção B — Supabase local via Docker (LAN)
5. Configurar o frontend
6. Rodar em desenvolvimento
7. Deploy em produção (Vercel / Netlify)
8. Configurar acesso LAN
9. Primeiro acesso e troca de senha
10. Configurar sessões de votação
11. Solução de problemas (Tela em Branco)
12. Backup e restauração

# 1. Visão geral da arquitetura

O sistema é composto por três camadas:

| <b>Camada</b>       | <b>Tecnologia</b>                | <b>Onde roda</b>                  |
|---------------------|----------------------------------|-----------------------------------|
| Frontend            | React + Vite                     | Navegador do eleitor (ou telão)   |
| Backend             | Supabase (Postgres + Auth + RPC) | Cloud (Supabase) ou Docker local  |
| Hospedagem estática | Vercel / Netlify / Docker        | Internet (Vercel) ou LAN (Docker) |

## Como o eleitor vota

- Acessa a URL do sistema (ex: votacao.exemplo.com)
- Vê a tela de boas-vindas com a lista de sessões
- Opcionalmente confirma a geolocalização (1x por sessão)
- Vota em cada sessão — candidatos + brancos, total pré-definido
- Recebe o comprovante **VT-YYYYMMDD-XXXXXX** ao concluir
- Pode baixar o PDF do comprovante direto no dispositivo

## Como o admin gerencia

- Acessa a URL com **#admin** ou clica 5× no título ou no botão "Acesso administrativo"
- Digita a senha (padrão: admin123)
- Cadastra sessões e candidatos na aba "Sessões"
- Acompanha resultados e auditoria em tempo real

## 2. Requisitos do sistema

### Para deploy em internet (Supabase Cloud + Vercel)

- Conta gratuita em **supabase.com**
- Conta gratuita em **vercel.com** ou **netlify.com**
- Git instalado (para deploy automático)
- Node.js 18+ (apenas para build/teste local)

### Para deploy em LAN (Docker local)

| <b>Componente</b>              | <b>Mínimo</b>                    | <b>Recomendado</b>          |
|--------------------------------|----------------------------------|-----------------------------|
| Servidor (PC/Mini-PC/Servidor) | 2 GB RAM, 2 vCPU, 20 GB disco    | 4 GB RAM, 4 vCPU, 50 GB SSD |
| Docker                         | 20.10+                           | 24.0+                       |
| Docker Compose                 | v2 (plugin)                      | v2.20+                      |
| Node.js (opcional, para dev)   | 18 LTS                           | 20 LTS                      |
| Sistema operacional            | Linux, macOS ou Windows com WSL2 | Ubuntu 22.04 LTS            |

### Para os eleitores

- Navegador moderno (Chrome, Edge, Firefox, Safari)
- Conexão à internet (ou Wi-Fi da LAN local)
- Permissão de geolocalização (se a urna exigir)

## 3. Opção A — Supabase Cloud (internet)

### 3.1. Criar projeto no Supabase

- Acesse <https://supabase.com> e crie uma conta gratuita
- Clique em **"New project"**
- Escolha um nome (ex: votacao-igreja)
- Defina uma senha forte para o banco de dados (guarde-a!)
- Selecione a região mais próxima (South America - São Paulo)
- Aguarde 1-2 minutos enquanto o projeto é provisionado

### 3.2. Criar as tabelas e funções

- No menu lateral, clique em **SQL Editor**
- Clique em **"New query"**
- Abra o arquivo `supabase/migrations/0001_init.sql` do projeto, copie todo o conteúdo e cole no editor
- Clique em **"Run"** (ou Ctrl+Enter) — deve aparecer `"Success. No rows returned"`

■ **Dica:** Este script é idempotente — pode rodar mais de uma vez sem erro.

- Crie outra query, abra `supabase/seed/seed.sql`, cole e execute. Isso cria uma eleição de exemplo com senha **admin123**.

### 3.3. Pegar as credenciais

- Vá em **Settings** → **API** no menu lateral
- Copie o valor de **Project URL** (algo como `https://abc123.supabase.co`)
- Copie o valor de **anon public key** (JWT longo que começa com `eyJ...`)

■■ **Atenção:** A chave **service\_role** NUNCA deve ser colocada no frontend. Use apenas a **anon**.

### 3.4. Pegar o ID da eleição

- Volte ao SQL Editor
- Rode a query: `SELECT id, name FROM public.elections;`
- Copie o UUID retornado — este é o **VITE\_ELECTION\_ID**

## 4. Opção B — Supabase local via Docker (LAN)

### 4.1. Instalar Docker

Instale Docker Desktop (Windows/macOS) ou Docker Engine (Linux):

- Windows/Mac: <https://www.docker.com/products/docker-desktop/>
- Linux (Ubuntu/Debian): `sudo apt install docker.io docker-compose-v2`

■ **Atenção:** No Windows, ative o WSL2 primeiro: `wsl --install` em PowerShell como admin.

### 4.2. Configurar variáveis de ambiente

Na raiz do projeto, copie o arquivo de exemplo:

```
cp .env.example .env
# Edite o .env com seu editor favorito
```

Edite as variáveis principais:

```
VITE_SUPABASE_URL=http://localhost:8000
VITE_SUPABASE_ANON_KEY=cole-sua-chave-aqui
VITE_ELECTION_ID=obtido-apos-seed

POSTGRES_PASSWORD=uma-senha-forte
JWT_SECRET=uma-string-aleatoria-de-pelo-menos-32-caracteres
```

### 4.3. Subir a stack completa

Use o docker-compose que já vem no projeto:

```
docker compose up -d
```

Aguarde 1-2 minutos enquanto os containers inicializam. Acompanhe com:

■ **Dica:** Na primeira vez, as tabelas são criadas automaticamente a partir de `supabase/migrations/0001_init.sql` e os dados de exemplo de `supabase/seed/seed.sql`.

### 4.4. Verificar o ID da eleição

Se você usou o seed, rode:

```
docker compose exec db psql -U postgres -d postgres -c 'SELECT id, name FROM public.elections ;'
```

Copie o UUID retornado e cole em `VITE_ELECTION_ID` no arquivo `.env`.

Reinicie o frontend:

```
docker compose restart frontend
```

### 4.5. Acessar os serviços

| <b>Serviço</b>  | <b>URL</b>                          | <b>Descrição</b>                   |
|-----------------|-------------------------------------|------------------------------------|
| Frontend        | <code>http://localhost:5173</code>  | Página de votação                  |
| Studio          | <code>http://localhost:54323</code> | Interface visual do banco de dados |
| API REST        | <code>http://localhost:8000</code>  | API do Supabase (Kong)             |
| Postgres direto | <code>localhost:5432</code>         | Para psql, DBeaver, etc.           |
| Mail catcher    | <code>http://localhost:54324</code> | Caixa de e-mail (Inbucket)         |

## 5. Configurar o frontend

### 5.1. Variáveis de ambiente

O frontend lê três variáveis no build/serve:

```
VITE_SUPABASE_URL=https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGc...sua-chave-aqui
VITE_ELECTION_ID=550e8400-e29b-41d4-a716-446655440000
```

### 5.2. Como obter cada uma

#### VITE\_SUPABASE\_URL

- Cloud: copie de Settings → API → Project URL
- Local: use `http://SEU_IP_LAN:8000` (não use localhost se for acessar de outro dispositivo)
- Local só nesse PC: use `http://localhost:8000`

#### VITE\_SUPABASE\_ANON\_KEY

- Cloud: copie de Settings → API → Project API keys → **anon** public
- Local (Docker): rode `docker compose exec db psql -U postgres -d postgres -t -c "SELECT anon_key FROM config"`

■■ **Atenção:** Use SEMPRE a chave **anon**. A chave **service\_role** é segredo do backend.

#### VITE\_ELECTION\_ID

- Rode no SQL Editor: `SELECT id FROM public.elections LIMIT 1;`
- Copie o UUID (formato: 8-4-4-4-12 caracteres hexadecimais)

## 6. Rodar em desenvolvimento

Para testar localmente sem Docker:

```
npm install
npm run dev
```

A aplicação abrirá em **http://localhost:5173**.

### 6.1. Testar o fluxo completo

- Acesse **http://localhost:5173** — deve aparecer a tela de boas-vindas
- Como admin: vá para **http://localhost:5173/#admin**
- Digite a senha **admin123**
- Vá em **Sessões** e cadastre:
  - - Título: "Presbíteros", Votos obrigatórios: 3, candidatos: João, Maria, Pedro, Ana
  - - Título: "Diáconos", Votos obrigatórios: 2, candidatos: Carlos, Paula, Marcos, Lúcia
- Volte para a página inicial (sem o #admin)
- Vote em cada sessão — confirme a localização se a urna exigir
- Receba o comprovante final **VT-...**
- Volte ao admin → aba **Resultados** para ver gráficos
- Aba **Auditoria** deve listar o comprovante emitido

### 6.2. Build de produção

```
npm run build
# Gera a pasta dist/ pronta para hospedagem estática
```

Para testar o build local:

```
npm run preview
```



## 7. Deploy em produção (Vercel / Netlify)

### 7.1. Deploy na Vercel (recomendado)

- Suba o projeto para o GitHub:

```
git init
git add .
git commit -m "Sistema de votacao"
git remote add origin https://github.com/seu-usuario/votacao-supabase.git
git push -u origin main
```

- Acesse <https://vercel.com> e importe o repositório
- A Vercel detecta automaticamente que é um projeto Vite
- Em **Environment Variables**, adicione:

```
VITE_SUPABASE_URL = https://seu-projeto.supabase.co
VITE_SUPABASE_ANON_KEY = eyJhbGc...
VITE_ELECTION_ID = uuid-da-eleicao
```

- Clique em **Deploy**
- Em ~1 minuto você terá uma URL pública (algo como votacao.vercel.app)

### 7.2. Deploy na Netlify

- Acesse <https://app.netlify.com>
- Clique em "Add new site" → "Import an existing project"
- Conecte ao GitHub e selecione o repositório
- Configurações de build:

```
Build command: npm run build
Publish directory: dist
```

- Adicione as mesmas 3 variáveis de ambiente
- Clique em **Deploy**

### 7.3. Deploy em hospedagem própria (Apache/Nginx)

Rode `npm run build` e suba o conteúdo de `dist/` para o servidor.

#### Configuração Nginx:

```
server {
    listen 80;
    server_name votacao.exemplo.com;
    root /var/www/votacao/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

#### Configuração Apache (.htaccess):

```
RewriteEngine On
RewriteBase /
RewriteRule ^index\.html$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

## 8. Configurar acesso LAN

Para que os dispositivos na mesma rede local consigam votar:

### 8.1. Descobrir o IP do servidor

- Windows: ipconfig (procure IPv4)
- Linux/macOS: ip addr ou ifconfig

Exemplo: **192.168.0.100**

### 8.2. Configurar o frontend

Edite o `.env` do frontend:

```
VITE_SUPABASE_URL=http://192.168.0.100:8000
```

Substitua **192.168.0.100** pelo IP do seu servidor.

### 8.3. Configurar o Kong (gateway Supabase)

Edite `supabase/config.toml`:

```
[api]
port = 8000
# O Kong precisa estar escutando em 0.0.0.0, não 127.0.0.1
```

Se necessário, ajuste o `docker-compose.yml` para o Kong:

```
kong:
  ports:
    - "8000:8000"
  environment:
    KONG_LISTEN: 0.0.0.0:8000
```

### 8.4. Firewall

Libere as portas no firewall do servidor:

```
# Linux (ufw)
sudo ufw allow 5173/tcp    # Frontend
sudo ufw allow 8000/tcp    # Supabase API
sudo ufw allow 5432/tcp    # Postgres (opcional, só se for acessar externamente)
sudo ufw reload
```

### 8.5. Testar de outro dispositivo

- Conecte o celular/tablet na mesma rede Wi-Fi
- Abra o navegador e vá para **http://192.168.0.100:5173**
- A tela de boas-vindas deve aparecer

## 9. Primeiro acesso e troca de senha

### 9.1. Acessar o painel admin

- Clique 5 vezes no título "**Urna Eletrônica**" na tela inicial
- Ou clique em "**Acesso administrativo**" no rodapé
- Ou acesse diretamente **http://localhost:5173/#admin**
- Digite a senha padrão **admin123**

### 9.2. Trocar a senha (obrigatório em produção)

- No painel, vá em **Geral**
- Role até "**Alterar Senha**"
- Digite a senha atual (**admin123**) e a nova senha (mín. 4 caracteres)
- Clique em **Alterar**

■■ **Atenção:** A senha é armazenada com hash + salt no banco. Não é possível recuperá-la — guarde bem!

### 9.3. Configurar local de votação

- Em **Geral** → **Local de Votação**:
- Nome do local (ex: Salão de Eventos)
- Latitude e longitude — clique em "**Usar minha localização atual**" para capturar automaticamente
- Raio permitido em metros (ex: 100m)
- Marque/desmarque "**Exigir geolocalização**" conforme sua necessidade
- Clique em "**Salvar Configurações**"

## 10. Configurar sessões de votação

### 10.1. Criar uma nova sessão

- No painel, clique na aba **Sessões**
- Clique em "**Nova Sessão**"
- Preencha:
  - - **Título:** ex: "Presbíteros"
  - - **Votos obrigatórios:** quantos o eleitor deve preencher
  - - **Candidatos:** um nome por linha (até 10)
- Clique em **Salvar**

### 10.2. Reordenar sessões

- Use os botões de seta ↑↓ ao lado de cada sessão
- A ordem define a sequência de votação para o eleitor

### 10.3. Editar/excluir sessão

- Botão de lápis (✎) para editar
- Botão de lixeira (🗑) para excluir — confirmação dupla

■■ **Atenção:** Excluir uma sessão remove também todos os votos daquela sessão. Use com cuidado!

### 10.4. Exemplo de configuração completa

Para uma eleição de liderança eclesial:

- **Sessão 1:** Presbíteros — 3 votos — 5 candidatos
- **Sessão 2:** Diáconos — 2 votos — 4 candidatos
- **Sessão 3:** Tesoureiros — 1 voto — 2 candidatos

## 11. Solução de problemas

### "Não foi possível obter sua localização"

- Verifique se o navegador tem permissão de geolocalização para o site
- HTTPS é necessário em produção — localhost também é aceito
- No painel admin, desative "**Exigir geolocalização**" se o local for difícil de georreferenciar
- Em ambiente interno, o GPS pode falhar — use coordenadas manuais ou desative

### "Você já votou nesta sessão"

- O token do navegador (localStorage) está marcado como tendo votado
- Para resetar: painel admin → aba **Segurança** → "**Zerar TUDO**"
- Ou em outro dispositivo: abra o link em uma janela anônima

### "Senha incorreta" no admin

- Tente a senha padrão: **admin123**
- Se você alterou e esqueceu, rode no SQL:

```
-- ATENÇÃO: isto reseta a senha para admin123
UPDATE public.elections
SET admin_password_hash = encode(digest('admin123' || admin_password_salt, 'sha256'), 'hex')
WHERE id = 'SEU-ELECTION-ID';
```

### Frontend em branco / erro de carregamento

- Abra o console do navegador (F12) e veja os erros
- Verifique se as 3 variáveis **VITE\_\*** estão corretas no .env
- Faça **Ctrl+F5** para forçar recarga sem cache
- Limpe o cache do Vite: `rm -rf node_modules/.vite && npm run dev`

### Porta 8000 já em uso (Docker)

```
# Ver o que está usando
sudo lsof -i :8000
# Matar o processo ou mudar a porta no docker-compose.yml
# de "8000:8000" para "8001:8000"
```

### Erro "Network Error" no frontend

- Verifique se o Supabase está rodando: `docker compose ps`
- Veja os logs: `docker compose logs kong`
- Verifique se o **VITE\_SUPABASE\_URL** está acessível do navegador

### Resultados não aparecem

- Abra a aba **Resultados** no admin — ela recarrega ao ser clicada
- Verifique no Studio do Supabase se há linhas em **votes**
- Confirme que o **VITE\_ELECTION\_ID** corresponde à eleição dos votos

## 11.1. PROBLEMA: Página em branco após deploy

■ ■ **Atenção:** Se a página de votação aparece completamente em branco, este guia vai te ajudar a diagnosticar e resolver em 5 minutos. A versão atual do sistema já mostra uma tela de erro amigável quando falta configuração — se você ainda vê página em branco, há outro problema abaixo.

## Causa raiz mais comum: variáveis de ambiente ausentes

O sistema precisa de 3 variáveis de ambiente para funcionar. Se faltar QUALQUER UMA, o frontend não consegue se conectar ao Supabase e a página fica em branco.

### As 3 variáveis obrigatórias

```
VITE_SUPABASE_URL=https://seu-projeto-supabase-co  
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

■ ■ **Atenção:** As variáveis DEVEM começar com VITE\_ — caso contrário o Vite não as expõe ao frontend!

## Diagnóstico passo a passo

### Passo 1: Verifique se a tela de erro amigável está aparecendo

Abra a URL do deploy. Se aparecer uma caixa azul com "**Configuração necessária**", o sistema já detectou o problema — siga para o Passo 2.

### Passo 2: Confirme as variáveis no painel da plataforma

Vercel: **Settings** → **Environment Variables**

Netlify: **Site settings** → **Environment variables**

### Passo 3: Faça um NOVO deploy

Após adicionar/alterar variáveis, é necessário um novo deploy. A Vercel e a Netlify NÃO recompilam automaticamente quando você muda variáveis de ambiente.

```
# Vercel: clique em Redeploy no aba Deployments  
# Netlify: clique em Trigger deploy - Deploy site
```

### Passo 4: Abra o console do navegador (F12)

Você deve ver uma mensagem como:

```
[Urna Eletrônica] { env: { url: OK, key: OK, election: OK }, ready: true }
```

Se algum campo estiver AUSENTE, volte ao Passo 2.

### Passo 5: Valide o VITE\_ELECTION\_ID

Abra o SQL Editor do Supabase e rode:

```
SELECT id, name FROM public.elections;
```

Copie EXATAMENTE o UUID retornado. Não use o UUID de exemplo 00000000-0000-0000-0000-000000000000!

## Causas menos comuns

### Build falhou silenciosamente

Veja os logs de build na Vercel/Netlify. Para testar localmente:

```
npm install && npm run build
```

### CORS / Mixed Content

Se o frontend está em HTTPS mas o Supabase em HTTP, o navegador bloqueia. Garanta que ambos usam HTTPS em produção.

### Seed não foi executado

Se você criou o Supabase mas esqueceu de rodar o seed, a tabela elections está vazia. Rode o seed no SQL Editor.

### Cache do navegador

Tente **Ctrl+Shift+R** (ou Cmd+Shift+R no Mac) para forçar recarga sem cache.

## Checklist final

- ■ As 3 variáveis VITE\_\* estão configuradas no painel?
- ■ Os valores não são os de exemplo (URL não é localhost, KEY não é eyJ...SUA\_CHAVE)?
- ■ O VITE\_ELECTION\_ID é um UUID válido que existe no banco?
- ■ O seed foi executado (a tabela elections tem pelo menos 1 linha)?
- ■ O deploy foi feito após alterar as variáveis?
- ■ O console do navegador não tem erros em vermelho?
- ■ O cache foi limpo (Ctrl+Shift+R)?

## 12. Backup e restauração

### 12.1. Backup do banco (Supabase Cloud)

No painel do Supabase:

- Vá em **Settings** → **Database**
- Clique em **"Daily Backups"** (plano gratuito: 7 dias de retenção)

### 12.2. Backup do banco (Docker local)

Faça dump do PostgreSQL:

```
docker compose exec db pg_dump -U postgres postgres \
| gzip > backup-$(date +%Y%m%d).sql.gz
```

Para restaurar:

```
gunzip -c backup-20260728.sql.gz | \
docker compose exec -T db psql -U postgres postgres
```

### 12.3. Backup apenas dos resultados (CSV)

No Studio do Supabase, vá em cada tabela e use **"Export to CSV"**:

- votes → todos os votos individuais
- voter\_completions → todos os comprovantes de sessão
- election\_receipts → comprovantes finais da eleição

### 12.4. Migração de dados

Para mover de Supabase local para Cloud (ou vice-versa):

- Faça dump: pg\_dump (passo 12.2)
- Restaure no destino: psql
- Atualize as variáveis **VITE\_SUPABASE\_URL** e **VITE\_SUPABASE\_ANON\_KEY** no frontend
- Faça novo build e deploy



# Apêndice A — Comandos úteis

## Docker

```
# Ver status dos containers
docker compose ps

# Ver logs em tempo real
docker compose logs -f

# Reiniciar tudo
docker compose restart

# Parar tudo
docker compose down

# Apagar TUDO (incluindo banco)
docker compose down -v

# Reconstruir imagens
docker compose build --no-cache

# Entrar no Postgres
docker compose exec db psql -U postgres -d postgres
```

## Frontend (npm)

```
# Instalar dependências
npm install

# Rodar dev (http://localhost:5173)
npm run dev

# Build de produção (pasta dist/)
npm run build

# Testar o build (http://localhost:4173)
npm run preview

# Limpar cache
rm -rf node_modules/.vite
npm run dev
```

## SQL úteis

```
-- Listar todas as eleições
SELECT id, name, location_name, created_at FROM public.elections;

-- Total de votos por sessão
SELECT
  vs.title,
  COUNT(*) FILTER (WHERE v.is_blank = false) AS votos_validos,
  COUNT(*) FILTER (WHERE v.is_blank = true) AS brancos
FROM voting_sessions vs
LEFT JOIN votes v ON v.session_id = vs.id
GROUP BY vs.id, vs.title;

-- Comprovantes emitidos
SELECT receipt_code, created_at
FROM election_receipts
ORDER BY created_at DESC;

-- Resetar senha do admin (cuidado!)
UPDATE elections
```

```
SET admin_password_hash = encode(digest('admin123' || admin_password_salt, 'sha256'), 'hex')  
WHERE id = 'COLE-0-UUID-AQUI';
```

## Apêndice B — Estrutura do banco de dados

| <b>Tabela</b>     | <b>Função</b>                                            |
|-------------------|----------------------------------------------------------|
| elections         | Configuração geral da eleição (nome, local, senha admin) |
| voting_sessions   | Sessões de votação (Presbíteros, Diáconos...)            |
| candidates        | Candidatos de cada sessão                                |
| votes             | Cada voto individual (para auditoria)                    |
| voter_completions | Sessões concluídas por cada eleitor + receipt da sessão  |
| election_receipts | Comprovante final emitido ao concluir todas as sessões   |

### Funções RPC disponíveis:

| <b>Função</b>                          | <b>Uso</b>                                                   |
|----------------------------------------|--------------------------------------------------------------|
| get_public_election(uuid)              | Ler config pública da eleição                                |
| get_voter_status(uuid, text)           | Status do eleitor (sessões concluídas)                       |
| submit_vote(...)                       | Submeter voto (com validações de geo, contagem, duplicidade) |
| finalize_election(uuid, text)          | Emitir comprovante final ao concluir todas as sessões        |
| verify_admin(uuid, text)               | Verificar senha do admin                                     |
| admin_update_election(...)             | Admin: atualizar config geral                                |
| admin_create_session(...)              | Admin: criar sessão                                          |
| admin_update_session(...)              | Admin: editar sessão                                         |
| admin_delete_session(uuid, text, uuid) | Admin: excluir sessão                                        |
| admin_change_password(...)             | Admin: trocar senha                                          |
| admin_get_results(uuid, text)          | Admin: resultados de todas as sessões                        |
| admin_list_receipts(uuid, text)        | Admin: lista de comprovantes emitidos                        |
| admin_reset_session(...)               | Admin: zerar votos de uma sessão                             |
| admin_reset_all(uuid, text)            | Admin: zerar tudo                                            |

## Suporte

Para dúvidas ou problemas:

- Abra uma issue no GitHub
- Consulte a documentação oficial do Supabase: **<https://supabase.com/docs>**
- Documentação do Vite: **<https://vitejs.dev>**

<fim do guia>

Gerado em 28/07/2026